TCS Data Engineer Interview Questions

1. How do you design an end-to-end data pipeline in a real project?

Answer:
I start by understanding the source systems (DB, API, files), data volume, and latency needs. Then I design:

- **Ingestion** (batch or streaming)
- **Storage layer** (Data Lake → Bronze/Silver/Gold)
- **Processing** (Spark / SQL / ADF)
- **Consumption** (BI, downstream apps)

I also include **logging, monitoring, retries, and alerting** from day one.

TCS checks whether you think **beyond coding** and focus on **production readiness**.

2. How do you handle incremental data loads when no timestamp column exists?

Answer:
I use:

- **Hash-based comparison** of full rows
- Or maintain a **snapshot table** and compare current vs previous data
- Sometimes a **surrogate key + checksum**

This avoids reprocessing full data and ensures only changed records are loaded.

This is a **very common TCS scenario question**.

3. Difference between ETL and ELT? What do you use in cloud projects?

Answer:

- **ETL**: Transform before loading (traditional systems)
- **ELT**: Load first, transform later (cloud-native)

In cloud projects (Azure / AWS / GCP), I mostly use **ELT** because storage is cheap and compute is scalable.

Interviewers want **practical choice**, not definition.

4. How do you optimize a slow Spark job?

Answer:
I check:

- Data skew
- Shuffle operations
- Incorrect joins (broadcast vs shuffle)
- Partition size

Then apply:

- Repartitioning
- Broadcast joins
- Caching only when required
- Avoiding wide transformations

TCS tests **debugging mindset**, not just Spark syntax.

5. How do you manage data quality in pipelines?

Answer:
I enforce:

- Null checks
- Schema validation
- Duplicate detection
- Record count reconciliation

Failed records go to **quarantine tables** with proper error reasons.

This shows **enterprise-level thinking**.

6. What is the difference between batch and streaming pipelines?

Answer:

- **Batch**: Scheduled, large data, cost-effective
- **Streaming**: Near real-time, event-based, low latency

I choose based on **business SLA**, not trend.

TCS often asks **when to use what**, not definitions.

7. How do you handle schema evolution?

Answer:
I use:

- Schema versioning
- Auto-merge (Delta / Iceberg)
- Backward compatibility checks

Breaking schema changes are handled through **controlled releases**.

8. How do you ensure pipeline reliability?

Answer:

- Idempotent loads
- Retry mechanisms
- Checkpointing
- Alerts via monitoring tools

If a job fails, it should **restart safely without data loss**.

9. What is data partitioning and why is it important?

Answer:
Partitioning improves:

- Query performance
- Parallel processing
- Cost optimization

I partition based on **date, region, or business keys** depending on access patterns.

10. How do you debug a failed production pipeline?

Answer:
I:

1. Check logs and error messages
2. Identify source vs transformation issue
3. Validate input data
4. Re-run using recovery logic

I never rerun full pipelines blindly.

This question tests **real on-call experience**.

11. How do you handle large files (100GB+)?

Answer:

- Split files
- Use distributed processing (Spark)
- Optimize partitions
- Avoid small file problems

12. SQL question TCS frequently asks:

"How do you find duplicate records?"

Answer:

Using GROUP BY with HAVING COUNT(*) > 1, and then deciding whether to delete or archive duplicates.

13. What is slowly changing dimension (SCD)? Which type have you used?

Answer:

I've mostly used:

- **SCD Type 1** – overwrite
- **SCD Type 2** – history with start/end dates

Type 2 is common in reporting systems.

14. How do you secure sensitive data?

Answer:

- Column-level masking
- Encryption at rest & in transit
- Role-based access
- Separate secure zones

Security awareness matters at TCS.

15. What tools have you used in real projects?

Typical Answer:

- SQL, Spark, Python
- Azure Data Factory / Databricks
- Data Lake / Warehouse
- Git, CI/CD

They care more about **how** you used them.